



INT305 Machine Learning

Lecture 9

Probabilistic Models

Siyue Yu

Department of Intelligence Science

siyue.yu02@xjtlu.edu.cn

Today

- So far in the course we have adopted a modular perspective, in which the model, loss function, optimizer, and regularizer are specified separately. *概率解释*
- Today we will begin putting together a **probabilistic interpretation** of the choice of model and loss, and introduce the concept of **maximum likelihood estimation**.
- Let's start with a simple biased coin example.

➤ You flip a coin $N = 100$ times and get outcomes $\{x_1, \dots, x_N\}$ where $x_i \in \{0, 1\}$ and $x_i = 1$ is interpreted as heads H . *抛硬币 100 次, $x_i=1$ 时代表正*

➤ Suppose you had $N_H = 55$ heads and $N_T = 45$ tails.

➤ What is the probability it will come up heads if we flip again? Let's design a model for this scenario, fit the model. We can use the fit model to predict the next outcome. *根据概率来预测下一次抛出的朝向。*

Model? 模型构建

- The coin is possibly loaded. So, we can assume that one coin flip outcome x is a **Bernoulli random variable** for some currently unknown parameter $\theta \in [0, 1]$.
 单次结果为伯努利随机变量 $p(x=1|\theta)=\theta$ and $p(x=0|\theta)=1-\theta$ 概率质量函数
 or more succinctly $p(x|\theta) = \theta^x(1-\theta)^{1-x}$ 未知参数 θ , 代表正面出现的真实概率。
- It's sensible to assume that $\{x_1, \dots, x_N\}$ are **independent and identically distributed (i.i.d.)** Bernoullis. 假设 N 次抛硬币的结果是独立同分布的。这是一个标准的简化计算的统计假设。
- Thus the joint probability of the outcome $\{x_1, \dots, x_N\}$ is

$$p(x_1, \dots, x_N | \theta) = \prod_{i=1}^N \theta^{x_i} (1 - \theta)^{1-x_i}$$

基于 i.i.d. 假设, 观察到整个序列 x 的联合概率是每次伯努利试验概率的乘积。

Loss? 损失函数与准则

- We call the probability mass (or density for continuous) of the observed data the **likelihood function** (as a function of the parameters θ): 似然函数: 当我们把联合概率 $p(x|\theta)$ 视为关于参数 θ 的函数时, 它被称为 likelihood function $L(\theta)$.

$$L(\theta) = \prod_{i=1}^N \theta^{x_i} (1 - \theta)^{1-x_i}$$
 观察到数据 x 为固定值, θ 是要选择的变量。
- We usually work with **log-likelihoods**: 对数似然 $\ell(\theta)$.
 取对数将连乘转化为相加。

$$\ell(\theta) = \sum_{i=1}^N x_i \log \theta + (1 - x_i) \log(1 - \theta)$$

- How can we choose θ ? Good values of θ should assign high probability to the observed data. This motivates the **maximum likelihood criterion**, that we should pick the parameters that maximize the likelihood:

找到使对数似然函数 $\ell(\theta)$ 达到最大值的参数 $\hat{\theta}_{ML}$.

$$\hat{\theta}_{ML} = \max_{\theta \in [0, 1]} \ell(\theta)$$

最大似然准则:

应选择最大程度解释已观测数据的参数 θ 。好的 θ 应该使观察到的数据出现的概率最高。

Maximum Likelihood Estimation for the Coin Example

- Remember how we found the optimal solution to linear regression by setting derivatives to zero? We can do that again for the coin example.

求解最大似然估计 $\hat{\theta}_{ML}$

对目标函数 (对数似然) 关于参数 θ 求导, 然后将导数设为零,

$$\frac{d\ell}{d\theta} = \frac{d}{d\theta} \left(\sum_{i=1}^N x_i \log \theta + (1 - x_i) \log(1 - \theta) \right)$$

先将对数似然 $\ell(\theta)$ 中的 $\sum x_i$ 替换为 N_H (正面次数), 将 $\sum (1 - x_i)$ 替换为 N_T (反面次数).

$$= \frac{d}{d\theta} (N_H \log \theta + N_T \log(1 - \theta))$$

$$= \frac{N_H}{\theta} - \frac{N_T}{1 - \theta}$$

求导 $\frac{d\ell}{d\theta} = \frac{N_H}{\theta} - \frac{N_T}{1 - \theta}$

where $N_H = \sum_i x_i$ and $N_T = N - \sum_i x_i$. $\therefore$ 导数为0, $\therefore \frac{N_H}{\theta} = \frac{N_T}{1 - \theta}$

- Setting this to zero gives the maximum likelihood estimate:

求解: $\hat{\theta}_{ML} = \frac{N_H}{N_H + N_T}$

得到正面概率 $\hat{\theta}_{ML}$ 正是我们观察到的正面的概率.

Maximum Likelihood Estimation

- Notice, in the coin example we are actually minimizing cross-entropies!

最大似然估计的通用框架:

步骤一: 定义一个能够为数据集分配概率 (或概率密度) 的概率模型.

步骤二: 通过最大化似然 (或最小化负对数似然) 来确定模型的参数.

$$\hat{\theta}_{ML} = \max_{\theta \in [0,1]} \ell(\theta)$$

交叉熵: 最小化交叉熵 = 交叉熵极大

结论: 这表明我们过去在神经网络中最小化的交叉熵损失, 从概率角度来看, 就是在进行最大似然估计.

$$= \min_{\theta \in [0,1]} -\ell(\theta)$$

$$= \min_{\theta \in [0,1]} \sum_{i=1}^N -x_i \log \theta - (1 - x_i) \log(1 - \theta)$$

MLE 提供了一个统一的、坚实的概率基础, 来解释许多机器学习算法的目标函数。

- This is an example of **maximum likelihood estimation**.
 - define a model that assigns a probability (or has a probability density at) to a dataset
 - maximize the likelihood (or minimize the neg. log-likelihood).
- Many examples we've considered fall in this framework! Let's consider classification again.

Discriminative VS Generative?

判别式和生成式

Two approaches to classification:

- **Discriminative approach**: estimate parameters of decision boundary/class separator directly from labeled examples.
 - Model $p(t|\mathbf{x})$ directly (logistic regression models)
 - Learn mappings from inputs to classes (linear/logistic regression, decision trees etc)
 - Tries to solve: How do I separate the classes?
- **Generative approach**: model the distribution of inputs characteristic of the class (Bayes classifier).
 - Model $p(\mathbf{x}|t)$
 - Apply Bayes Rule to derive $p(t|\mathbf{x})$
 - Tries to solve: What does each class "look" like?
- Key difference: is there a distributional assumption over inputs?

第一张幻灯片：判别式 vs 生成式 (Discriminative VS Generative)

这张图区分了解决分类问题的两种核心策略：

1. 判别式方法 (Discriminative Approach)：

- 核心目标：直接估计决策边界（或类别分隔符）的参数。
- 模型关注点：直接建模目标 t 在给定输入 $\mathbf{x}$ 下的概率，即 $p(t|\mathbf{x})$ 。
- 解决问题：“我如何将类别分开？”
- 典型模型：逻辑回归、决策树、支持向量机 (SVM) 等。

2. 生成式方法 (Generative Approach)：

- 核心目标：为每个类别（目标 t ）的输入数据 $\mathbf{x}$ 建模。
- 模型关注点：建模输入 $\mathbf{x}$ 在给定目标 t 下的概率，即 $p(\mathbf{x}|t)$ 。
- 分类机制：通过应用贝叶斯法则 (Bayes Rule) 将 $p(\mathbf{x}|t)$ 转换为所需的 $p(t|\mathbf{x})$ 。
- 解决问题：“每个类别的数据看起来像什么？”
- 典型模型：贝叶斯分类器 (Bayes classifier)。

3. 关键区别：

- 生成式方法通常需要对输入数据 $\mathbf{x}$ 的分布做出更强的概率假设，而判别式方法通常不需要。

AGenerativeModel:BayesClassifier生成式模型示例 (A Generative Model: Bayes Classifier)

- Aim to classify text into spam/not-spam (yes $c=1$; no $c=0$)
- Example: “You are one of the very few who have been selected as a winners for the free \$1000 Gift Card.”
- Use bag-of-words features, get binary vector $\mathbf{x}$ for each email
- Vocabulary:
 - “a”: 1
 - ...
 - “car”: 0
 - “card”: 1
 - ...
 - “win”: 0
 - “winner”: 1
 - “winter”: 0
 - ...
 - “you”: 1

第二张幻灯片：生成式模型示例 (A Generative Model: Bayes Classifier)

这张图为我们将要学习的贝叶斯分类器设定了一个具体的应用场景。

1. 分类任务：
 - 目标：将邮件文本分类为垃圾邮件 (spam, $c = 1$) 或非垃圾邮件 (not-spam, $c = 0$)。
2. 特征表示 (Feature Engineering):
 - 方法：使用词袋模型 (Bag-of-Words, BoW) 特征。
 - 数据结构：将每封邮件表示为一个二元向量 $\mathbf{x}$ 。
 - 向量 $\mathbf{x}$ 的构造：
 - 向量的每个维度对应词汇表中的一个单词。
 - 如果该单词出现在邮件中，对应维度的值 $x_i = 1$ 。
 - 如果该单词未出现在邮件中，对应维度的值 $x_i = 0$ 。
 - 示例：在“您被选为 \$1000 礼品卡的得奖者”的邮件中，“you”和“winner”对应的 x_i 为 1，而“car”和“winter”对应的 x_i 为 0。

BayesClassifier

- Given features $\mathbf{x} = [x_1, x_2, \dots, x_D]^T$ we want to compute class probabilities using Bayes Rule:

$$\underbrace{p(c|\mathbf{x})}_{\text{Pr. class given words}} = \frac{p(\mathbf{x}, c)}{p(\mathbf{x})} = \frac{\overbrace{p(\mathbf{x}|c)}^{\text{Pr. words given class}} p(c)}{p(\mathbf{x})}$$

- More formally

$$\text{posterior} = \frac{\text{Class likelihood} \times \text{prior}}{\text{Evidence}}$$

- How can we compute $p(\mathbf{x})$ for the two-class case? (Do we need to?)

$$p(\mathbf{x}) = p(\mathbf{x}|c=0)p(c=0) + p(\mathbf{x}|c=1)p(c=1)$$

- To compute $p(c|\mathbf{x})$ we need: $p(\mathbf{x}|c)$ and $p(c)$

贝叶斯分类器的数学基础"贝叶斯法则", 这是我们从生成式模型 $p(\mathbf{x}|c)$ 转向分类目标 $p(c|\mathbf{x})$

1. 贝叶斯法则的构成

我们希望计算在给定特征 $\mathbf{x}$ 的情况下, 类别 c 发生的概率 $p(c|\mathbf{x})$, 贝叶斯法则定义了计算公式:

$$p(c|\mathbf{x}) = \frac{p(\mathbf{x}|c)p(c)}{p(\mathbf{x})}$$

- $p(c|\mathbf{x})$ (Posterior / 后验概率):

- 含义: 这是我们想要的结果。在观察到特征 $\mathbf{x}$ 之后, 类别为 c 的概率。

- $p(\mathbf{x}|c)$ (Class Likelihood / 类似然):

- 含义: 生成式模型的核心。在类别 c 下, 出现特征 $\mathbf{x}$ 的概率。

- $p(c)$ (Prior / 先验概率):

- 含义: 在未观察到任何特征 $\mathbf{x}$ 之前, 类别 c 发生的固有概率。

- $p(\mathbf{x})$ (Evidence / 证据):

- 含义: 观察到特征 $\mathbf{x}$ 的总概率。它是一个归一化常数。

2. 证据 $p(\mathbf{x})$ 的计算与必要性

- 如何计算 $p(\mathbf{x})$?

- 对于二分类问题 ($c=0$ 和 $c=1$), $p(\mathbf{x})$ 必须通过全概率公式 (Law of Total Probability) 来计算:

- 它本质上是所有类别似然与先验概率乘积的总和。

- 是否需要计算 $p(\mathbf{x})$?

- 不需要。对于分类任务, 我们的目标是找到使 $p(c|\mathbf{x})$ 最大的那个类别 $\hat{c}$:

- 由于 $p(\mathbf{x})$ 是一个不依赖于类别 c 的常数, 它不会影响最大化的选择。

- 因此, 我们只需要最大化分子即可!

3. 分类所需的要素

为了用贝叶斯法则进行分类, 我们需要具备两个核心信息:

- $p(\mathbf{x}|c)$ (类似然): 每个类别的数据分布模型。

- $p(c)$ (先验概率): 每个类别的基发生概率。

结论: 贝叶斯分类器的主要任务, 就是找到合理的模型来估计 $p(\mathbf{x}|c)$ 和 $p(c)$ 。

NaiveBayes

- Assume we have two classes: spam and non-spam. We have a dictionary of D words, and binary features $\mathbf{x} = [x_1, x_2, \dots, x_D]$ saying whether each word appears in the e-mail.
- If we define a joint distribution $p(c, x_1, \dots, x_D)$, this gives enough information to determine $p(c)$ and $p(\mathbf{x}|c)$.
- Problem: specifying a joint distribution over $D + 1$ binary variables requires 2^{D+1} entries. This is computationally prohibitive and would require an absurd amount of data to fit.
- We'd like to impose **structure** on the distribution such that:
 - it can be **compactly** represented
 - learning** and **inference** are both tractable

NaiveBayes

- Naive assumption: **Naive Bayes** assumes that the word features x_i are **conditionally independent** given the class c .
 - This means x_i and x_j are independent under the conditional distribution $p(\mathbf{x}|c)$.
 - Note: this doesn't mean they're independent.
 - Mathematically,

$$p(c, x_1, \dots, x_D) = p(c)p(x_1|c) \cdots p(x_D|c).$$

- Compact representation of the joint distribution
 - Prior probability of class: $p(c = 1) = \pi$ (e.g. spam email)
 - Conditional probability of word feature given class: $p(x_j = 1|c) = \theta_{jc}$ (e.g. word "price" appearing in spam)
 - $2D + 1$ parameters total (before 2^{D+1} — 1)

第一张幻灯片：计算复杂性问题的 (The Problem)

1. 模型设定： $\mathbf{x} = [x_1, \dots, x_D]$

- 任务是垃圾邮件分类。有两个类别（垃圾邮件/非垃圾邮件）。
- 输入特征 $\mathbf{x}$ 是一个长度为 D (词汇表大小) 的二元向量，表示单词是否出现。
- 总共有 $D + 1$ 个二元变量需要建模 (D 个词特征和 1 个类别特征 c)。

2. 计算障碍：

- 为了构建一个完整的生成式分类器，我们需要确定联合分布 $p(c, x_1, \dots, x_D)$ 。
- 问题：指定一个包含 $D + 1$ 个二元变量的联合分布，需要 $2^{D+1} - 1$ 个独立参数。
- 结论：如果词汇表 D 很大 (例如 $D = 10,000$)，这个参数数量是无法计算和联合的。

3. 解决方案的目标：

- 我们必须对这个分布添加结构 (structure)。
- 目标是：① 使模型能够紧凑地表示，② 使学习和推断过程易于处理 (tractable)。

第二张幻灯片：朴素假设与参数简化 (The Naive Assumption)

这张幻灯片引入了解决上述问题的核心假设。

1. 朴素假设 (Naive assumption)：

- 朴素贝叶斯假设：给定类别 c 时，所有的词特征 x_i 都是条件独立的。
- 数学表示： $p(x_1, x_2 | c) = p(x_1 | c)p(x_2 | c)$ 。
- 含义：这个假设忽略了语言的内在结构 (例如，给定一封邮件是垃圾邮件，那么单词“免费”的出现不影响单词“中奖”的出现)，尽管这个假设在现实中往往是错误的，但它是使模型成立的唯一途径。

2. 联合分布的紧凑表示：

- 由于条件独立性，原本复杂的联合概率简化为：

- 整个联合分布被简化成了单个类先验概率和 D 个词特征的条件概率的乘积。

3. 参数数量的缩减：

- 先验概率 (Prior)：只需要一个参数 $\pi = p(c = 1)$ 。
- 条件概率 (Likelihood)：需要 $2D$ 个参数 (D 个词在 $c = 1$ 下的概率，和 D 个词在 $c = 0$ 下的概率)。
- 总参数：从不可计算的 $2^{D+1} - 1$ 个，大幅减少到可处理的 $2D + 1$ 个。

BayesNets

- We can represent this model using an **directed graphical model**, or **Bayesian network**:

节点 c 代表类别变量 (如垃圾/非垃圾邮件)。

节点 $x_1, x_2, \dots, x_D$ 代表特征变量 (词汇表中各个单词)。

箭头从节点 c 指向所有特征 x_i ，在贝叶斯网络中，箭头代表条件依赖关系， x_i 的概率取决于父节点 c 。

我们可以使用有向图模型 (directed graphical model) 或贝叶斯网络来直观地表示模型中的变量关系。

一般规则：任何贝叶斯网络图结构都意味着其联合分布可以分解为所有变量的条件概率的乘积。条件是该变量的父节点。

类别 c 没有父节点，所以 $p(c)$ 先验。

每个特征 x_i 的唯一父节点是 c ，所以 $p(x_i | c)$ 类似然。

$p(c, x_1, \dots, x_D) = p(c) \cdot p(x_1 | c) \cdot p(x_2 | c) \cdot \dots \cdot p(x_D | c)$

- This graph structure means the joint distribution factorizes as a product of conditional distributions for each variable given its parent(s).

- Intuitively, you can think of the edges as reflecting a causal structure. But mathematically, this doesn't hold without additional assumptions.

直觉 (Intuition)：从直觉上，你可以将这些箭头理解为一种因果结构。例如，类别 c (“这是垃圾邮件”) 是导致特征 x_i (“出现 free 这个词”) 出现的原因。

数学严谨性：然而，从数学上讲，贝叶斯网络仅仅依赖于条件独立性假设来简化联合分布。模型并不需要严格的因果关系来成立。

NaiveBayes:Learning

- The parameters can be learned efficiently because the log-likelihood decomposes into independent terms for each feature.

$$\begin{aligned}
 \ell(\theta) &= \sum_{i=1}^N \log p(c^{(i)}, \mathbf{x}^{(i)}) = \sum_{i=1}^N \log \{p(\mathbf{x}^{(i)} | c^{(i)}) p(c^{(i)})\} \\
 &= \sum_{i=1}^N \log \left\{ p(c^{(i)}) \prod_{j=1}^D p(x_j^{(i)} | c^{(i)}) \right\} \\
 &= \sum_{i=1}^N \left[\log p(c^{(i)}) + \sum_{j=1}^D \log p(x_j^{(i)} | c^{(i)}) \right] \\
 &= \underbrace{\sum_{i=1}^N \log p(c^{(i)})}_{\text{Bernoulli log-likelihood of labels}} + \underbrace{\sum_{j=1}^D \sum_{i=1}^N \log p(x_j^{(i)} | c^{(i)})}_{\text{Bernoulli log-likelihood for feature } x_j}
 \end{aligned}$$

- Each of these log-likelihood terms depends on different sets of parameters, so they can be optimized independently.

NaiveBayes:Learning

- We can handle these terms separately. For the prior we maximize: $\sum_{i=1}^N \log p(c^{(i)})$

This is a minor variant of our coin flip example. Let $p(c^{(i)} = 1) = \pi$. Note $p(c^{(i)}) = \pi^{c^{(i)}} (1 - \pi)^{1 - c^{(i)}}$

- Log-likelihood:

$$\sum_{i=1}^N \log p(c^{(i)}) = \sum_{i=1}^N c^{(i)} \log \pi + \sum_{i=1}^N (1 - c^{(i)}) \log(1 - \pi)$$

- Obtain MLEs by setting derivatives to zero:

$$\hat{\pi} = \frac{\sum_i \mathbb{I}[c^{(i)} = 1]}{N} = \frac{\# \text{ spams in dataset}}{\text{total } \# \text{ samples}}$$

NaiveBayes: Learning

- Each θ_{jc} 's can be treated separately: maximize $\sum_{i=1}^N \log p(x_j^{(i)} | c^{(i)})$
- This is (again) a minor variant of our coin flip example. Let $\theta_{jc} = p(x_j^{(i)} = 1 | c)$. Note

$$p(x_j^{(i)} | c) = \theta_{jc}^{x_j^{(i)}} (1 - \theta_{jc})^{1 - x_j^{(i)}}.$$

- Log-likelihood:

$$\begin{aligned} \sum_{i=1}^N \log p(x_j^{(i)} | c^{(i)}) &= \sum_{i=1}^N c^{(i)} \left\{ x_j^{(i)} \log \theta_{j1} + (1 - x_j^{(i)}) \log(1 - \theta_{j1}) \right\} \\ &\quad + \sum_{i=1}^N (1 - c^{(i)}) \left\{ x_j^{(i)} \log \theta_{j0} + (1 - x_j^{(i)}) \log(1 - \theta_{j0}) \right\} \end{aligned}$$

- Obtain MLEs by setting derivatives to zero:

$$\hat{\theta}_{jc} = \frac{\sum_i \mathbb{I}[x_j^{(i)} = 1 \ \& \ c^{(i)} = c]}{\sum_i \mathbb{I}[c^{(i)} = c]} \quad \text{for } c = 1 \quad \frac{\# \text{word } j \text{ appears in spams}}{\# \text{ spams in dataset}}$$

第一张幻灯片：对数似然的分解 (Decomposition of Log-Likelihood)

这张幻灯片展示了 Naive Bayes 学习高效性的数学基础。

- 初始目标：我们希望最大化观察到整个数据集 $\mathcal{D}$ 的联合对数似然 $\ell(\theta) = \sum \log p(c^{(i)}, \mathbf{x}^{(i)})$ 。
- 核心分解：由于朴素贝叶斯假设（特征在给定类别下条件独立），总的对数似然 $\ell(\theta)$ 可以被分解为两个独立的项：

$$\ell(\theta) = \underbrace{\sum_{i=1}^N \log p(c^{(i)})}_{\text{项 1: 类别先验}} + \underbrace{\sum_{j=1}^D \sum_{i=1}^N \log p(x_j^{(i)} | c^{(i)})}_{\text{项 2: 特征似然}}$$

- 学习效率：这种分解是至关重要的，因为它表明：

- 项 1 只依赖于类标签 c 的参数（先验概率）。
- 项 2 只依赖于特征和类别的参数（条件概率 θ_{jc} ）。
- 由于这两项依赖于不同的参数集，因此我们可以将一个复杂的优化问题拆解成两个（或 $2D + 1$ 个）独立的、易于求解的小问题。

第二张幻灯片：学习类别先验 (Learning the Prior $p(c)$)

这张幻灯片展示了如何求解第一个独立项（类别先验）。

- 目标：最大化 $\sum \log p(c^{(i)})$ 。这又回到了我们之前学习的硬币抛掷例子。
- 参数：我们要求解的参数是 π ，即类别 $c = 1$ 的先验概率 π 。
- MLE 结果：通过对对数似然求导并设为零，得到的最大似然估计（MLE）是：

Maximum Likelihood Estimation

$$\hat{\pi} = \frac{\sum_i \mathbb{I}[c^{(i)} = 1]}{N} = \frac{\text{数据集中垃圾邮件的数量}}{\text{总样本数量}}$$

- 结论：类别先验概率的 MLE 就是该类别在训练集中出现的观察频率。

NaiveBayes:Inference 推断

- We predict the category by performing **inference** in the model.
- Apply **Bayes' Rule**:

$$p(c | \mathbf{x}) = \frac{p(c)p(\mathbf{x} | c)}{\sum_{c'} p(c')p(\mathbf{x} | c')} = \frac{p(c) \prod_{j=1}^D p(x_j | c)}{\sum_{c'} p(c') \prod_{j=1}^D p(x_j | c')}$$

- We need not compute the denominator if we're simply trying to determine the most likely c .
- Shorthand notation:

$$p(c | \mathbf{x}) \propto p(c) \prod_{j=1}^D p(x_j | c)$$

- For input $\mathbf{x}$, predict by comparing the values of $p(c) \prod_{j=1}^D p(x_j | c)$ for different c (e.g. choose the largest).

第一张幻灯片：推断过程 (Naive Bayes: Inference)

这张图展示了如何应用贝叶斯法则，并利用“朴素假设”来做出最终的分类决策。

1. 应用贝叶斯法则：

- 左侧是标准的贝叶斯法则：
- 右侧是应用了“朴素假设”后的展开式：
 - 分子 $p(\mathbf{x}|c)$ 被分解为 D 个独立特征概率的乘积： $\prod_{j=1}^D p(x_j|c)$ 。
 - 分母 $p(\mathbf{x})$ 也相应地被展开成对所有类别 c 的联合概率求和。

2. 分母的省略：

- 核心原则：我们不需要计算完整的后验概率 $p(c|\mathbf{x})$ ，只需要找到概率最大的那个类别 c 。
- 简化：由于分母 $\sum_c p(c)p(\mathbf{x}|c)$ 对于所有类别 c 都是一个常数（它不依赖于 c ），因此可以被省略。
- 简写符号：

3. 最终分类决策：

- 方法：对于一个新的输入 $\mathbf{x}$ ，我们只需计算 $p(c) \prod_{j=1}^D p(x_j|c)$ 的值，然后选择使得这个值最大的那个类别 c 。

NaiveBayes

- Naive Bayes is an amazingly cheap learning algorithm!
- **Training time:** estimate parameters using maximum likelihood
 - Compute co-occurrence counts of each feature with the labels.
 - Requires only one pass through the data!
- **Test time:** apply Bayes' Rule
 - Cheap because of the model structure. (For more general models, Bayesian inference can be very expensive and/or complicated.)
- We covered the Bernoulli case for simplicity. But our analysis easily extends to other probability distributions.
- Unfortunately, it's usually less accurate in practice compared to discriminative models due to its "naive" independence assumption.

朴素贝叶斯算法在计算效率和实际准确性方面的特点。

高效性 (An amazingly cheap learning algorithm!):

训练时间:

算法只需通过计算每个特征与标签的共现次数 (即计算频率)。

学习过程只需遍历数据一次 (one pass), 没有复杂的迭代优化过程 (如梯度下降), 因此训练速度极快。

测试时间:

由于模型结构简单 (只有简单的乘法和求和), 应用贝叶斯法则进行推断也非常快。

局限性 (Limitations):

准确性: 朴素贝叶斯在实践中的准确性通常不如判别式模型 (如逻辑回归、SVM 或神经网络)。

原因: 这是由于它的“朴素”假设 (特征在给定类别下是条件独立的) 导致的。这个假设在真实世界的复杂数据 (如语言) 中经常被违反, 这限制了模型的表达能力。

泛化性:

尽管我们用二元伯努利分布进行了分析, 但朴素贝叶斯的分析框架可以轻松扩展到其他概率分布 (例如高斯分布、多项式分布), 用于处理连续特征或计数特征。

总结: 朴素贝叶斯是一种计算极其高效的算法, 它通过将复杂问题简化为简单的频率计数来实现快速训练。然而, 它的准确性受限于“朴素”假设。

MLE Issue: Data Sparsity

核心问题：最大似然估计有一个缺陷——当训练数据量不足（数据稀疏）时，它会导致模型过度拟合。

- Maximum likelihood has a pitfall: if you have too little data, it can overfit.
- E.g., what if you flip the coin twice and get H both times? Eg. 假设我们只抛了两次硬币，并且都是正面

$$\theta_{\text{ML}} = \frac{N_H}{N_H + N_T} = \frac{2}{2 + 0} = 1$$

根据MLE公式 $\theta_{\text{ML}} = \frac{N_H}{N_H + N_T}$
 则正面概率为 $\theta_{\text{ML}} = \frac{2}{2+0} = 1$

- Because it never observed T, it assigns this outcome probability 0. This problem is known as **data sparsity**.

数据稀疏性的后果(The Consequence):

问题：因为模型从未观察到反面(Tails, T)，它根据 MLE 规则，将出现反面的概率估计为 0。

含义：模型从数据中得出了一个过于极端的结论：反面是绝对不可能发生的。

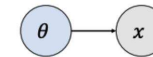
影响：

过度拟合：这是一种极端的过度拟合，模型完全信任了它观察到的有限数据。

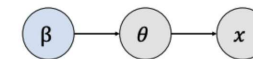
实际危害：如果我们在下一次测试中真的抛到了反面，模型给这个真实事件分配了 0 的概率，这会在后续的概率计算中（尤其是朴素贝叶斯中的概率乘法）导致严重错误（任何数乘以 0 都得 0），使得整个模型不可用。

Bayesian Parameter Estimation

- In maximum likelihood, the observations are treated as random variables, but the parameters are not.



- The **Bayesian** approach treats the parameters as random variables as well. β is the set of parameters in the prior distribution of θ .



- To define a Bayesian model, we need to specify two distributions:
 - The **prior distribution** $p(\theta)$, which encodes our beliefs about the parameters *before* we observe the data
 - The **likelihood** $p(\mathcal{D}|\theta)$, same as in maximum likelihood

BayesianParameterEstimation

- When we **update** our beliefs based on the observations, we compute the **posterior distribution** using Bayes' Rule:

$$p(\boldsymbol{\theta} | \mathcal{D}) = \frac{p(\boldsymbol{\theta})p(\mathcal{D} | \boldsymbol{\theta})}{\int p(\boldsymbol{\theta}')p(\mathcal{D} | \boldsymbol{\theta}') d\boldsymbol{\theta}'}$$

- We rarely ever compute the denominator explicitly. In general, it is computationally intractable.

BayesianParameterEstimation

- Let's revisit the coin example. We already know the likelihood:

$$L(\theta) = p(\mathcal{D} | \theta) = \theta^{N_H} (1 - \theta)^{N_T}$$

- It remains to specify the prior $p(\theta)$.
 - We can choose an **uninformative prior**, which assumes as little as possible. A reasonable choice is the uniform prior.
 - But our experience tells us 0.5 is more likely than 0.99. One particularly useful prior that lets us specify this is the beta distribution:

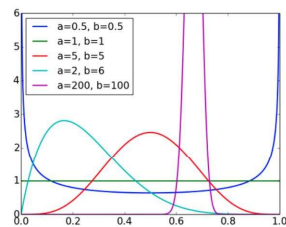
$$p(\theta; a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} \theta^{a-1} (1-\theta)^{b-1}.$$

- This notation for proportionality lets us ignore the normalization constant:

$$p(\theta; a, b) \propto \theta^{a-1} (1-\theta)^{b-1}.$$

BayesianParameterEstimation

- Beta distribution for various values of a, b :



- Some observations:
 - The expectation $\mathbb{E}[\theta] = a/(a + b)$ (easy to derive).
 - The distribution gets more peaked when a and b are large.
 - The uniform distribution is the special case where $a = b = 1$.
- The beta distribution is used for as a prior for the Bernoulli distribution.

BayesianParameterEstimation

- Computing the posterior distribution: $P(\theta) = P(\theta; a, b)$

$$\begin{aligned} p(\theta | \mathcal{D}) &\propto p(\theta)p(\mathcal{D} | \theta) \\ &\propto [\theta^{a-1}(1-\theta)^{b-1}] [\theta^{N_H}(1-\theta)^{N_T}] \\ &= \theta^{a-1+N_H}(1-\theta)^{b-1+N_T}. \end{aligned}$$

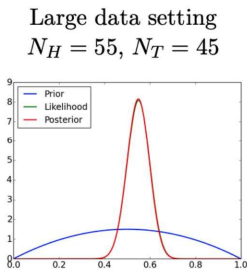
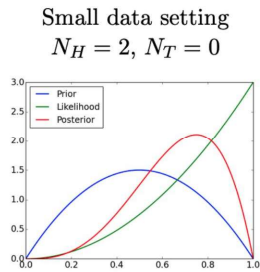
- This is just a beta distribution with parameters $N_H + a$ and $N_T + b$.
- The posterior expectation of θ is:

$$\mathbb{E}[\theta | \mathcal{D}] = \frac{N_H + a}{N_H + N_T + a + b}$$

- The parameters a and b of the prior can be thought of as **pseudo-counts**.
 - The reason this works is that the prior and likelihood have the same functional form. This phenomenon is known as **conjugacy** (conjugate priors), and it's very useful.

BayesianParameterEstimation

Bayesian inference for the coin flip example:



When you have enough observations, the data overwhelm the prior.

如何避免在数据太少时，做出极端的错误判断。

1. 问题的根源：“菜鸟法官” (MLE)

传统做法 (MLE): 我们之前学到的最大似然估计，可以看作是一个“菜鸟法官”。他只相信眼前看到的证据。

示例：你只抛了两次硬币，两次都是正面（2个正面，0个反面）。

判决：菜鸟法官会宣布：“根据我看到的证据，反面出现的概率是 0%！反面是不可能存在的！”

后果 (数据稀疏性): 这种极端的 0% 判决就是过度拟合。它把一个小概率事件当作了不可能事件。如果下次真的出现反面，整个模型的判断就会崩溃。

2. 贝叶斯方法的解决之道：“经验法官”

贝叶斯方法：可以看作是一个“经验丰富的老法官”。他相信证据，但他更相信他多年积累的“经验”（或称作“常识”）。

老法官的逻辑：他知道世界上几乎不存在完美的硬币，所以即使你只给他 2 个正面，他也不会立即宣布反面概率为 0。

3. 核心机制：添加“经验保险”

经验 (Prior): 在贝叶斯方法中，这种“经验”被称为先验分布。通过使用 Beta 分布作为先验，我们可以轻松地将其“经验”转化为数据。

伪计数 (Pseudo-Counts): 这种经验被量化为“伪计数”（比如 a 和 b）。你可以把它视为一个保险策略。

策略：在看你的真实数据之前，我先在账本上偷偷记下“我预计会有 a 次正面，和 b 次反面”。

老法官的最终判决：他会将你的真实观测数据 (N_H, N_T) 和他的经验伪计数 (a, b) 加在一起，然后做出最终判决。

4. 最终结果：避免极端

- 判决公式：

最终概率 = $\frac{\text{真实观测到的正面次数} + \text{经验正面计数}(a)}{N_H + N_T + a + b}$

1. 概念基础: MLE 与贝叶斯方法的根本区别

特点	最大似然估计 (MLE)	贝叶斯方法 (Bayesian)
对待参数 (θ)	视为固定的、未知参数。	视为随机变量，有自身的概率分布 $p(\theta)$ 。
所需操作	仅需要似然函数 $\text{Sp}(\text{mathcal{D}})$	Ytheta 。

2. 贝叶斯方法的两大组件

为了定义贝叶斯模型，我们需要指定以下两部分：

1. 先验分布 $\{p(\theta)\}$:

- 含义：我们在观察数据 $\mathcal{D}$ 之前，对参数 θ 的初始信念。
- 选择：对于伯努利分布的参数 θ (即正负概率)，统计学上通常选择 Beta 分布作为先验。Beta 分布由参数 a 和 b 决定。
- 优势 (共轭性)：选择 Beta 分布 (共轭先验) 的目的是为了让数学处理更加简便，因为它能保证后验分布的形式与先验分布相同。

2. 后验分布 $\{p(\theta|\mathcal{D})\}$:

- 目标：这是我们最终更新的信念。我们利用贝叶斯法则来计算后验分布 $p(\theta|\mathcal{D})$ ，即在看到数据 $\mathcal{D}$ 之后 θ 的概率分布。
- 法则： $p(\theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta)$ (后验 $\propto$ 似然 $\times$ 先验)。

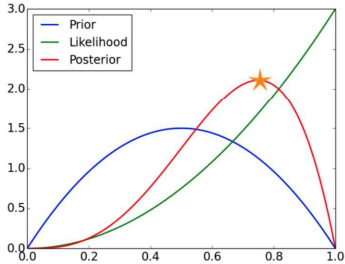
3. 最终估计与解决稀疏性问题

贝叶斯方法的最终参数估计值，是后验分布的期望 (平均值)，而不是后验分布的峰值 (如 MLE)。

- 后验计算：
 - 将 Beta 先验与伯努利似然相乘后，得到的后验分布仍然是一个 Beta 分布，但参数已经更新：
 - 新的参数：由 $\text{Beta}(N_F + a, N_T + b)$ 决定。
- 贝叶斯估计值：
 - 贝叶斯估计值是后验分布的期望 $E[\theta|\mathcal{D}]$ 。
- 解决数据稀疏性 (伪计数)：
 - 关键点：先验参数 a 和 b 被视为伪计数 (pseudo-counts)。

MaximumA-PosterioriEstimation

Maximum a-posteriori (MAP) estimation: find the most likely parameter settings under the posterior



Maximum A-Posteriori Estimation

- This converts the Bayesian parameter estimation problem into a maximization problem

$$\begin{aligned}
 \hat{\theta}_{\text{MAP}} &= \arg \max_{\theta} p(\theta | \mathcal{D}) \\
 &= \arg \max_{\theta} p(\theta, \mathcal{D}) \\
 &= \arg \max_{\theta} p(\theta) p(\mathcal{D} | \theta) \\
 &= \arg \max_{\theta} \log p(\theta) + \log p(\mathcal{D} | \theta)
 \end{aligned}$$

Maximum A-Posteriori Estimation

- Joint probability in the coin flip example:

$$\begin{aligned}
 \log p(\theta, \mathcal{D}) &= \log p(\theta) + \log p(\mathcal{D} | \theta) \\
 &= \text{Const} + (a - 1) \log \theta + (b - 1) \log(1 - \theta) + N_H \log \theta + N_T \log(1 - \theta) \\
 &= \text{Const} + (N_H + a - 1) \log \theta + (N_T + b - 1) \log(1 - \theta)
 \end{aligned}$$

- Maximize by finding a critical point

$$0 = \frac{d}{d\theta} \log p(\theta, \mathcal{D}) = \frac{N_H + a - 1}{\theta} - \frac{N_T + b - 1}{1 - \theta}$$

- Solving for θ ,

$$\hat{\theta}_{\text{MAP}} = \frac{N_H + a - 1}{N_H + N_T + a + b - 2}$$

Maximum A-Posteriori Estimation

Comparison of estimates in the coin flip example:

	Formula	$N_H = 2, N_T = 0$	$N_H = 55, N_T = 45$
$\hat{\theta}_{ML}$	$\frac{N_H}{N_H + N_T}$	1	$\frac{55}{100} = 0.55$
$\mathbb{E}[\theta \mathcal{D}]$	$\frac{N_H + a}{N_H + N_T + a + b}$	$\frac{4}{6} \approx 0.67$	$\frac{57}{104} \approx 0.548$
$\hat{\theta}_{MAP}$	$\frac{N_H + a - 1}{N_H + N_T + a + b - 2}$	$\frac{3}{4} = 0.75$	$\frac{56}{102} \approx 0.549$

$\hat{\theta}_{MAP}$ assigns nonzero probabilities as long as $a, b > 1$.

1. MAP 估计的定义与目标 (Slides 3 & 4)

1. 定义：最大后验估计 (MAP estimation) 是一种寻找后随分布 $p(\theta | \mathcal{D})$ 峰值的参数估计方法。

2. 目标：找到最能解释数据 $\mathcal{D}$ 并且最符合我们先验信念的参数 $\hat{\theta}_{MAP}$ 。

$$\hat{\theta}_{MAP} = \underset{\theta}{\operatorname{argmax}} p(\theta | \mathcal{D})$$

3. 优化函数的转换：

- 利用贝叶斯法则 $p(\theta | \mathcal{D}) \propto p(\mathcal{D} | \theta) p(\theta)$ ，并将乘积转化为对数求和（因为对数是单调函数，不影响最大值位置）。

MAP 的目标函数是：

$$\hat{\theta}_{MAP} = \underset{\theta}{\operatorname{argmax}} [\log p(\theta) + \log p(\mathcal{D} | \theta)]$$

- 核心区别：这表明 MAP 的优化目标是（对数先验）+（对数似然）。它在最大化数据拟合度的同时，也考虑了先验信念。

2. MAP 的计算 (Slide 5)

在硬币抛掷的例子中（使用 Beta 先验和伯努利似然）：

1. 目标函数：将 Beta 分布的 $\log p(\theta)$ 和伯努利分布的 $\log p(\mathcal{D} | \theta)$ 代入上面的 MAP 目标函数。

2. 求解：通过对简化后的对数联合概率 $\log p(\theta, \mathcal{D})$ 求导，并设为零（找到临界点）。

3. 结果：求解得到的 MAP 估计值是：

$$\hat{\theta}_{MAP} = \frac{N_H + a - 1}{N_H + N_T + a + b - 2}$$

3. 三种参数估计方法的比较 (Slide 2)

这张表对比了我们在概率模型中学习的三种参数估计方法：

方法	目标	公式特点	2 正面, 0 反面 ($a = 1, b = 1$)	鲁棒性处理
MLE ($\hat{\theta}_{ML}$)	最大化似然 (数据证据)	只包含 N_H 和 N_T	1	失败 (结果为 1, 不合理)